

# Лабораторная работа #2

## “Построение графа потока управления”

### Цель работы

Реализовать модуль для построения графа потока управления программы на основе синтаксического дерева разбора.

### Описание реализации

**Структуры данных** Реализованы следующие структуры данных для представления информации:

1. **LocalVar** - информация о локальной переменной:

- name - имя переменной
- type - тип переменной
- offset - смещение в стеке

2. **Argument** - информация о параметре функции:

- name - имя параметра
- type - тип параметра
- index - индекс параметра

3. **BasicBlock** - базовый блок графа потока управления:

- id - уникальный идентификатор
- label - метка блока
- operations - операции в блоке
- successors - список последователей
- predecessors - список предшественников
- line\_start, line\_end - начальная и конечная строки
- is\_visited - флаг посещения

4. **Function** - информация о функции:

- name - имя функции
- return\_type - тип возвращаемого значения
- args - список аргументов
- local\_vars - список локальных переменных
- source\_file - имя исходного файла
- entry\_block - входной блок
- exit\_block - выходной блок

5. **CFG** - граф потока управления:

- `function` - информация о функции
- `blocks` - список базовых блоков
- `block_count` - количество блоков
- `next_block_id` - следующий доступный идентификатор блока

**Алгоритм построения графа** Алгоритм построения графа потока управления реализован в функции `build_cfg_from_function`:

1. Извлекается информация о функции (имя, тип возвращаемого значения, параметры)
2. Извлекается информация о локальных переменных
3. Создаются начальный и конечный блоки
4. Рекурсивно обрабатываются операторы функции:
  - Для оператора `if` создается структура с блоками для условия, `then`-ветви, `else`-ветви и слияния
  - Для оператора `while` создается структура с блоками для условия, тела цикла и выхода
  - Для оператора `do-while` создается аналогичная структура
  - Для простых операторов создается отдельный базовый блок

**Вывод графа** Реализована функция `print_cfg_dot` для вывода графа в формате DOT, которая:

- Выводит информацию о функции (имя, тип возвращаемого значения)
- Выводит информацию о параметрах и локальных переменных
- Выводит все базовые блоки с их операциями
- Выводит связи между блоками

Также реализована функция `print_call_graph_dot` для построения графа вызовов функций.

## Результаты тестирования

**Тест 1: Факториал** Исходный код:

```
int factorial(int n) {
    int result;
    result = 1;
    while (n > 1) {
        result = result * n;
        n = n - 1;
    }
}
```

Граф потока управления: - Блок 0 (entry): объявление переменной `result` - Блок 2: присваивание `result = 1` - Блок 3: цикл `while (n > 1)` - Блок 1 (exit): выход из функции

### Тест 2: Максимум двух чисел    Исходный код:

```
int max(int a, int b) {  
    int result;  
    if (a > b) {  
        result = a;  
    } else {  
        result = b;  
    }  
}
```

Граф потока управления: - Блок 0 (entry): объявление переменной result - Блок 2: оператор if(a > b) - Блок 1 (exit): выход из функции

### Тест 3: Сложная функция    Исходный код:

```
int complex_function(int x, int y) {  
    int result;  
    int temp;  
  
    result = 0;  
  
    if (x > y) {  
        result = x * 2;  
    } else {  
        result = y + 1;  
        while (result < 10) {  
            result = result + 1;  
        }  
    }  
  
    return result;  
}
```

Граф потока управления: - Блок 0 (entry): объявление переменной result - Блок 2: объявление переменной temp - Блок 3: присваивание result = 0 - Блок 4: оператор if (x > y) с вложенным циклом while - Блок 5: объявление возвращаемой переменной result - Блок 1 (exit): выход из функции

### Выводы

Реализованный модуль успешно строит графы потока управления для различных конструкций языка программирования, включая: - Простые операторы присваивания - Условные операторы if-else - Циклы while и do-while - Вложенные конструкции

Модуль корректно обрабатывает информацию о функциях, их параметрах и локальных переменных, а также строит граф вызовов функций.